

UNIDAD 6 · SEMANA 9

Señales y memoria compartida

📅 Semana 9 · fechas por confirmar </> Laboratorio TD5 (resuelto + desafío IA)

~ 3 animaciones

LABORATORIO TD5 · RESUELTO

Señales y memoria compartida (mmap , shm)

Dos formas de comunicar procesos sin tuberías: avisarse con señales y compartir páginas de memoria. Juntas permiten que un proceso deje un mensaje en memoria y "toque el timbre" del otro.

✓ Informe del laboratorio

Entrega este laboratorio con un informe corto en LaTeX, con la plantilla del curso: cada ejercicio explicado, el código que importa y lo que ejecutaste en la terminal. Puedes usar IA generativa para escribir el LaTeX: decláralo y verifica cada resultado. Plantilla, ejemplo y guía de Prism →

EJERCICIOS

- | | | | |
|---|---------------------------------|---|---|
| 1 | Capturar SIGUSR1 con sigaction | 4 | Compartir código: una "biblioteca" hecha a mano |
| 2 | Mensaje por mmap + señal | 5 | Memoria compartida System V |
| 3 | Protecciones de un mmap anónimo | | |

TD5 · 1

● Resuelto · probado

Capturar SIGUSR1 con sigaction(2)

Enunciado. mysignal muestra su pid, instala un manejador para SIGUSR1 que muestra el número de señal, espera señales y restaura el manejador por defecto. Prueba desde otra terminal con kill : SIGUSR1, la secuencia SIGSTOP → SIGUSR1 → SIGCONT, y SIGKILL.

Solución



C tds/td5/mysignal.c

```

/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
 * ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores
the default action
 * From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT
<pid>, kill -KILL <pid>
 * After the n-th SIGUSR1 the default action is back: the next SIGUSR1
terminates the process.
 */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be
interrupted mid-buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n',
0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 ||
write_all(STDERR_FILENO, digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;

    printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
    fflush(stdout);

    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);          /* nothing extra blocked while the
handler runs */

```

```
sa.sa_flags = 0;
exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

/* Block SIGUSR1 outside sigsuspend: a signal arriving between the test
of `received`
and the wait stays pending instead of being lost (the pause() race).
*/
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);    /* atomically: unblock SIGUSR1 and
sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &block, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}
```

Salida real

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P;
kill -USR1 $P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P;
echo "exit status: $? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

- **Con el proceso detenido (SIGSTOP)**, la SIGUSR1 no se pierde: queda *pendiente* y el manejador se ejecuta al recibir SIGCONT. SIGSTOP y SIGKILL no se pueden capturar ni bloquear: son la garantía del administrador.
- **El manejador solo usa funciones seguras en señales** (`write`, no `printf`): la señal puede llegar en medio de un `printf` del programa y reentrar en su búfer.
- `sigprocmask` + `sigsuspend` en lugar de `pause`: si la señal llega entre la prueba de `received` y `pause`, `pause` dormiría para siempre. Con la señal bloqueada, espera pendiente; `sigsuspend` la desbloquea y duerme en un solo paso atómico.

Animación A6.1 · Entrega de señales →

TD5 · 2

● Resuelto · probado

Un mensaje por `mmap` y una señal

Enunciado. `receiver` mapea un archivo de 128 bytes, escribe su pid al inicio y espera SIGUSR1; al recibirla muestra el mensaje. `sender` mapea el mismo archivo, lee el pid, copia su entrada estándar en la zona compartida y envía SIGUSR1. ¿Qué pasa si se escriben más de 128 caracteres? ¿De qué tamaño es realmente el segmento?

Solución

C tds/td5/receiver.c

```
/*
 * TD5 - exercise 2: receiver. Maps block.txt (shared), writes its pid at the
 * start of the
 * mapping, waits for SIGUSR1 and prints the message that the sender left
 * after the pid.
 * ./receiver &      then      echo "hello" | ./sender
 */
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "block.txt"
#define SIZE 128

static void on_usr1(int sig) { (void)sig; } /* only interrupts sigsuspend
*/

int main(void)
{
```

```

struct sigaction sa = { 0 };
sigset_t usr1, old, wait_mask;

/* 1. Block SIGUSR1 BEFORE publishing the pid: a sender can only signal
us after that,
    and the signal will wait (pending) until sigsuspend. */
sigemptyset(&usr1);
sigaddset(&usr1, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &usr1, &old) == -1, "sigprocmask");
sa.sa_handler = on_usr1;
sigemptyset(&sa.sa_mask);
exit_if(sigaction(SIGUSR1, &sa, NULL) == -1, "sigaction");

/* 2. The file must be at least SIZE bytes: mapping past the end of a
file gives SIGBUS. */
int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
exit_if(fd == -1, "open " FILENAME);
exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");
char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
exit_if(shared == MAP_FAILED, "mmap");
exit_if(close(fd) == -1, "close");          /* the mapping stays
valid */

/* 3. Publish our pid. */
pid_t me = getpid();
memcpy(shared, &me, sizeof me);
printf("receiver %d: waiting for SIGUSR1\n", me);
fflush(stdout);

/* 4. Sleep with SIGUSR1 unblocked, atomically. */
wait_mask = old;
sigdelset(&wait_mask, SIGUSR1);
sigsuspend(&wait_mask);
exit_if(sigprocmask(SIG_SETMASK, &old, NULL) == -1, "sigprocmask"); /*
SETMASK, not BLOCK */

shared[SIZE - 1] = '\0';                    /* never trust the
other side */

printf("received: %s", shared + sizeof me);
exit_if(munmap(shared, SIZE) == -1, "munmap");
return EXIT_SUCCESS;
}

```

C tds/td5/sender.c

```
int main(void)
{
    int fd = open(FILENAME, O_RDWR);
    exit_if(fd == -1, "open " FILENAME);
    char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
    exit_if(shared == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    pid_t pid;
    memcpy(&pid, shared, sizeof pid);
    printf("sender: the receiver is %d\n", pid);

    /* Copy stdin into the message area, keeping one byte for '\0'. */
    size_t off = sizeof pid, max = SIZE - 1;
    ssize_t n = 1;
    while (off < max && (n = read(STDIN_FILENO, shared + off, max - off)) >
0)
        off += (size_t)n;
    exit_if(n == -1, "read");
    shared[off] = '\0';

    exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also
push to the file */
    exit_if(kill(pid, SIGUSR1) == -1, "kill");
    exit_if(munmap(shared, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Salida real

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

C tds/td5/mmap_limits.c

```
/*
 * TD5 - exercise 2, last question: how big is the shared segment of a 128-
byte file?
 * The kernel maps whole pages. We map TWO pages of a 128-byte file and write
byte by byte:
```

```

*   bytes 0..127      belong to the file
*   bytes 128..4095   exist (same page) but are never written back to the
file
*   byte 4096         is in a page that lies entirely past the end of the
file: SIGBUS
* The signal is caught and siglongjmp brings us back to report where it
happened.
*/
#include <fcntl.h>
#include <setjmp.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "limits.txt"
#define FILESIZE 128

static sigjmp_buf env;
static volatile sig_atomic_t caught;
static char *volatile map;          /* volatile: must survive the
siglongjmp */
static volatile long offset;

static void on_fault(int sig)
{
    caught = sig;
    siglongjmp(env, 1);
}

int main(void)
{
    const long page = sysconf(_SC_PAGESIZE);
    int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open");
    exit_if(ftruncate(fd, FILESIZE) == -1, "ftruncate");
    map = mmap(NULL, 2 * page, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(map == MAP_FAILED, "mmap");

    struct sigaction sa = { 0 };
    sa.sa_handler = on_fault;
    sigemptyset(&sa.sa_mask);

```

```

exit_if(sigaction(SIGSEGV, &sa, NULL) == -1 || sigaction(SIGBUS, &sa,
NULL) == -1, "sigaction");

if (sigsetjmp(env, 1) == 0)
    for (offset = 0; offset < 2 * page; offset++)
        map[offset] = 'x';

printf("file size %d bytes, page size %ld bytes, mapping of %ld bytes\n",
FILESIZE, page, 2 * page);
printf("wrote bytes 0..%ld; byte %ld raised %s\n", offset - 1, offset,
caught ? signame(caught) : "nothing");

exit_if(munmap(map, 2 * page) == -1, "munmap");
off_t end = lseek(fd, 0, SEEK_END);          /* the file did not
grow */
exit_if(end == -1, "lseek");
printf("file size after the writes: %lld bytes\n", (long long)end);
exit_if(close(fd) == -1, "close");
return EXIT_SUCCESS;
}

```

Salida real

```

$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes

```

El tamaño real es una página (4096 bytes). El núcleo mapea páginas completas: los bytes 128 a 4095 existen en memoria pero no pertenecen al archivo (nunca se escriben en el disco y el archivo sigue midiendo 128 bytes). Una página que está entera más allá del final del archivo produce **SIGBUS** al tocarla. Por eso `receiver` hace `ftruncate` : mapear un archivo vacío y escribir en él también da SIGBUS.

⚠ Corregido

La versión heredada restauraba la máscara con `SIG_BLOCK` (no la restauraba: la agregaba), llamaba a `printf` dentro del manejador y el mensaje pisaba el pid publicado. Ahora el pid ocupa los primeros bytes y el mensaje va después.

TD5 · 3

● Resuelto · salidas reales

Protecciones de un `mmap` anónimo

Enunciado. `map_anonymous` pide un entero con `mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0)`, muestra el puntero, lee `*p`, escribe 42 y lo lee otra vez. Compíllalo con `PROT_NONE`, `PROT_READ`, `PROT_WRITE`, `PROT_EXEC` y `PROT_EXEC|PROT_WRITE`, y explica cada resultado con `mmap(2)`.

Solución



C tds/td5/map_anonymous.c

```

/*
 * TD5 - exercise 3: protections of an anonymous mapping.
 * PROT is given at compile time; the Makefile builds map_none, map_read,
map_write, map_exec
 * and map_ew (PROT_EXEC|PROT_WRITE). Each program reads *p, writes 42 and
reads it again.
 * Read mmap(2): on x86-64, PROT_WRITE implies read access; PROT_NONE forbids
everything.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#ifndef PROT
#define PROT (PROT_READ | PROT_WRITE)
#endif

/* Readable name of the protection flags, e.g. "PROT_EXEC|PROT_WRITE". */
static const char *prot_name(int prot)
{
    static char name[64];
    name[0] = '\0';
    if (prot == PROT_NONE)
        return "PROT_NONE";
    if (prot & PROT_READ) strcat(name, "|PROT_READ");
    if (prot & PROT_WRITE) strcat(name, "|PROT_WRITE");
    if (prot & PROT_EXEC) strcat(name, "|PROT_EXEC");
    return name + 1;
}

int main(void)
{
    int *p = mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1,
0);
    exit_if(p == MAP_FAILED, "mmap");

    fprintf(stderr, "PROT = %s\n", prot_name(PROT));
    fprintf(stderr, "pointer: %p\n", (void *)p);
    fprintf(stderr, "read:    *p = %d\n", *p);          /* anonymous memory

```

```
starts zeroed */
    *p = 42;
    fprintf(stderr, "write:  *p = %d\n", *p);

    exit_if(munmap(p, sizeof(int)) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Salida real

```
$ ./map_none
PROT = PROT_NONE
pointer: 0x742c16995000
bash: line 1: 357416 Segmentation fault      (core dumped) ./map_none
[exit status 139]
```

Salida real

```
$ ./map_read
PROT = PROT_READ
pointer: 0x7b6b0d393000
read:  *p = 0
bash: line 1: 357431 Segmentation fault      (core dumped) ./map_read
[exit status 139]
```

Salida real

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:  *p = 0
write: *p = 42
```

Salida real

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

Salida real

```
$ ./map_ew
PROT = PROT_WRITE|PROT_EXEC
pointer: 0x7872e8140000
read:    *p = 0
write:   *p = 42
```

La MMU aplica los permisos de cada página: `PROT_NONE` prohíbe incluso leer; `PROT_READ` permite leer (la memoria anónima empieza en cero) pero no escribir; en x86-64 `PROT_WRITE` implica lectura. `PROT_EXEC` sola es un caso interesante: en este equipo (con claves de protección de memoria) Linux crea memoria *solo ejecutable* y la lectura falla; en procesadores sin esa función, la misma prueba lee 0. Por eso `mmap(2)` dice que el efecto de cada combinación "depende de la arquitectura". Cada violación es un fallo de página que el núcleo convierte en SIGSEGV (estado 139).

TD5 · 4

● Resuelto · probado

Compartir código: una "biblioteca" hecha a mano

Enunciado. Simula una biblioteca compartida: `load_add` copia el código máquina de `add(int, int)` a un archivo de 2048 bytes mapeado con `MAP_SHARED`; `use_add` mapea ese archivo con `PROT_EXEC`, lo usa como puntero a función y llama `f(42, 12)`. ¿Qué pasa con varios `use_add` a la vez?

Solución



C tds/td5/load_add.c

```

int add(int a, int b)
{
    return a + b;
}

int main(void)
{
    int fd = open(LIBFILE, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open " LIBFILE);
    /* An empty file cannot be mapped usefully (access past its end =
    SIGBUS): give it its size. */
    exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");

    char *code = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    /* Copy the first bytes of add(). Converting a function pointer to void *
    is not ISO C,
        but POSIX requires it to work (dlsym relies on it). */
    memcpy(code, (const void *)add, 64);

    exit_if(munmap(code, SIZE) == -1, "munmap"); /* MAP_SHARED: the bytes
    are in the file */
    return EXIT_SUCCESS;
}

```

C tds/td5/use_add.c

```
int main(void)
{
    int fd = open(LIBFILE, O_RDONLY);
    exit_if(fd == -1, "open " LIBFILE);

    void *code = mmap(NULL, SIZE, PROT_READ | PROT_EXEC, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    int (*f)(int, int) = (int (*)(int, int))code;
    printf("f(42, 12) = %d\n", f(42, 12));

    exit_if(munmap(code, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Salida real

```
$ ./load_add && ./use_add
f(42, 12) = 54
```

Funciona porque `add` no usa direcciones absolutas: su código es independiente de la posición. Varios `use_add` comparten la misma página física (la de la caché de páginas del archivo), igual que todos los programas comparten las páginas de `libc.so`. Un cargador real (`ld.so`, `dlopen`) además resuelve símbolos y reubicaciones. La versión heredada mapeaba el archivo sin darle tamaño y terminaba en SIGBUS.

TD5 · 5

● Resuelto · probado

Memoria compartida System V

Enunciado. (1) Crea u obtén un segmento de 1024 bytes con `ftok` + `shmget`, adjúntalo con `shmat`, incrementa su primer entero, muéstralo y sepáralo con `shmdt`. Ejecútalo varias veces: ¿qué observas? (2) Con `fork` en un bucle de 4 vueltas, 16 procesos registran su pid en el segmento (número de pids + arreglo de pids) y el proceso inicial muestra la lista. Repite con 32 procesos.

Solución



C tds/td5/shm_counter.c

```

/*
 * TD5 - exercise 5: a System V shared memory segment that survives the
 processes.
 * ./shm_counter increments the first int of the segment and
 prints it
 * ./shm_counter --remove destroys the segment (IPC_RMID)
 * Run it several times: 1, 2, 3... The segment lives in the kernel until it
 is removed or
 * the machine reboots (list them with `ipcs -m`, remove with `ipcrm`).
 * The key comes from ftok(file, id): same file + same id = same key on this
 machine.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#include "utils.h"

#define KEYFILE "/etc/passwd" /* any file that exists on every Linux
 machine */
#define KEYID 'F'
#define SIZE 1024

int main(int argc, char *argv[])
{
    key_t key = ftok(KEYFILE, KEYID);
    exit_if(key == -1, "ftok");
    int id = shmget(key, SIZE, IPC_CREAT | 0644);
    exit_if(id == -1, "shmget");

    if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
        exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl IPC_RMID");
        printf("segment %d removed\n", id);
        return EXIT_SUCCESS;
    }

    int *counter = shmat(id, NULL, 0);
    exit_if(counter == (void *)-1, "shmat");
    counter[0]++; /* a fresh segment is zero-
 filled */

```

```
printf("segment %d (key 0x%08x): counter = %d\n", id, (unsigned)key,
counter[0]);
exit_if(shmdt(counter) == -1, "shmdt");
return EXIT_SUCCESS;
}
```

Salida real

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

C tds/td5/shm_fork.c

```
/*
 * TD5 - exercise 5 (end): 16 processes (4 fork() in a loop) register their
 * pid in a shared
 * segment: segment[0] = number of pids, then the pids. The original process
 * prints the list.
 *
 * "count++; table[count] = pid" is NOT atomic: two processes can read the
 * same count and
 * one registration is lost. Build with -DNO_ATOMIC (Makefile target
 * shm_race) to see it;
 * the default version reserves the slot with an atomic fetch-and-add (a
 * preview of unit 7).
 * The segment is IPC_PRIVATE: only this process and its descendants (which
 * inherit the
 * attachment through fork) can see it.
 */
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define FORKS 4 /* 2^4 = 16 processes */
#define SIZE 4096
```

```
int main(int argc, char *argv[])
{
    int forks = argc > 1 ? atoi(argv[1]) : FORKS;
    int id = shmget(IPC_PRIVATE, SIZE, IPC_CREAT | 0600);
    exit_if(id == -1, "shmget");
    int *seg = shmat(id, NULL, 0);
    exit_if(seg == (void *)-1, "shmat");
    /* Mark for deletion now: it disappears when the last process detaches
    (even on a crash). */
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl");

    pid_t original = getpid();
    int children = 0;
    for (int i = 0; i < forks; i++) {
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0)
            children = 0;          /* the child starts with no children
of its own */
        else
            children++;
    }

#ifdef NO_ATOMIC
    int slot = seg[0] + 1;          /* read ... */
    sched_yield();                 /* ... (widen the window) ... */
    seg[0] = slot;                 /* ... write: not atomic */
#else
    int slot = __atomic_add_fetch(&seg[0], 1, __ATOMIC_SEQ_CST); /* read-
modify-write in one step */
#endif
    if (slot < SIZE / (int)sizeof(int))
        seg[slot] = getpid();

    while (children-- > 0)
        exit_if(wait(NULL) == -1, "wait");

    if (getpid() == original) {
        printf("%d processes registered (expected %d):", seg[0], 1 << forks);
        for (int i = 1; i <= seg[0] && i < SIZE / (int)sizeof(int); i++)
            printf(" %d", seg[i]);
        printf("\n");
    }
    exit_if(shmdt(seg) == -1, "shmdt");
}
```

```
return EXIT_SUCCESS;
}
```

Salida real

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568
357571 357569 357565 357574 357573 357570 357576 357578 357575 357577 357579
```

Salida real

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

- **El segmento sobrevive a los procesos:** el contador sigue 1, 2, 3... porque el segmento vive en el núcleo hasta que alguien lo borra (`IPC_RMID` , `ipcrm`) o se reinicia la máquina. `ipcs -m` lo muestra. La versión heredada lo borraba en la primera ejecución y el fenómeno no se veía.
- **Con 32 procesos se pierden registros** (entre 16 y 25 de 32 en las cinco ejecuciones reales de arriba): `seg[0]++` es leer–sumar–escribir, y dos procesos pueden leer el mismo valor. Es una *condición de carrera*, el tema de la unidad 7. La solución usa un incremento atómico del procesador (`__atomic_add_fetch`).
- `ftok` usa un archivo que existe en todo Linux (`/etc/passwd`); el enunciado original usaba `/etc/bashrc` , que no existe en Ubuntu.

Lista de cotejo

Desafío con IA generativa Reemplazar ``pause`` por ``sigsuspend``... y demostrar la carrera

Objetivo. Escribir con la IA un programa que pierda una señal usando ``pause()`` (la carrera clásica), medir con qué frecuencia ocurre y demostrar que ``sigprocmask`` + ``sigsuspend`` la elimina.

Prompt inicial. Úsalo en la herramienta que prefieras (Claude, ChatGPT, Gemini, Copilot, Codex...) con la versión *resuelta* del TD como contexto. Pide primero un plan y revisa cada paso antes de aceptarlo.

Prompt

Quiero demostrar experimentalmente la carrera de `pause()`. Ayúdame a escribir: (1) un receptor que hace `while (!llego) pause();`` con un manejador que pone `llego = 1;` (2) un emisor que envía `SIGUSR1` justo después de que el receptor publica su `pid;` (3) un script que repite el experimento 1000 veces con un límite de 1 s y cuenta cuántas veces el receptor se queda dormido para siempre. Después hagamos la versión con `sigprocmask` + `sigsuspend` y comparemos. Explícame la ventana exacta en la que se pierde la señal.

Cómo verificar el resultado

- La versión con ``pause()`` se cuelga al menos algunas veces (si nunca pasa, agranda la ventana con un ``usleep`` antes de ``pause``).
- La versión con ``sigsuspend`` nunca se cuelga en 1000 repeticiones.
- Puedes señalar en el código la línea exacta entre la prueba y la espera.

Declara el uso de IA: herramienta, prompts relevantes, qué verificaste tú y qué corregiste. Lo que cuenta es que puedas explicar y comprobar cada decisión.